

A CASE-STUDY BASED REPORT
ON
Spam Email Classification Using Naive Bayes and Support Vector Machines

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING
BACHELOR OF TECHNOLOGY
in Computer Science and Engineering
A.Y.: 2026-27

by

K. Pranav **Roll No. 2520030477**

Abhay Tripathi **Roll No. 2520030353**

Under the Esteemed guidance of

Jyothsna Datti

Asst.Professor, Dept. of CSE



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

INDEX

Abstract.....	3
Introduction.....	3
Background.....	3
Motivation for the Project.....	3
Overview of the Work Carried Out.....	4
Scope and Responsible-Use Statement.....	4
Problem Statement.....	4
Case-Study Scenario.....	4
Central Case-Study Question.....	4
Formal Problem Definition.....	5
Objectives that Guide the Study.....	5
Assumptions and Limitations.....	5
Dataset and Exploratory Data Analysis.....	6
About the Dataset.....	6
Purpose of EDA.....	6
Preparation Steps Actually Performed.....	6
EDA Exploration - Implementation.....	7
Summary of EDA Findings.....	7
Algorithm Selection and Implementation.....	8
Conclusion.....	10

Abstract

This case study examines how a text-classification application can distinguish spam from legitimate messages while keeping its decisions understandable. Our project, Spam Email Classification Using Naive Bayes and Support Vector Machines, uses a labelled short-message corpus, TF-IDF feature extraction, Multinomial Naive Bayes, and baseline and tuned SVM classifiers. The application serves predictions through a Flask dashboard with model comparison, token evidence, message statistics, and kernel-study results.

The bundled dataset contains 5,572 raw messages. Its implemented cleaning procedure produces 5,169 records, comprising 4,516 ham and 653 spam messages. An 80/20 stratified split contains 4,135 training and 1,034 test records. On the saved experiment, Naive Bayes achieves 97.49% accuracy and 89.08% spam F1, linear SVM achieves 98.16% accuracy and 92.31% F1, and the selected RBF SVM achieves 98.07% accuracy and 91.94% F1. These results were checked by reproducing predictions from the packaged models.

The study shows that SVM detects more spam than Naive Bayes at the recorded decision boundaries, while Naive Bayes produces fewer false positives. It also shows that the tuned model does not necessarily outperform the baseline on the test partition. The report follows the supplied case-study structure and discusses actual data preparation, EDA, implementation, and the limitations of applying SMS-based results to complete emails.

Introduction

Background

Spam filtering is a supervised classification problem. A model learns from messages whose labels are known and applies the learned relationship to another message. Before classification, text must be converted into numbers because the selected estimators operate on numerical feature vectors. TF-IDF represents words and short word sequences by their occurrence in a message and their frequency across training messages.

Naive Bayes combines class-specific feature evidence with the class prior. SVM learns a decision boundary in the feature space. Comparing these approaches on the same data makes it possible to observe differences in the mistakes they make. This comparison is more useful than reporting only whether one pasted message is labelled SPAM or HAM.

Motivation for the Project

For students, an incorrectly blocked academic message can matter as much as an unwanted message that passes through a filter. Our project therefore combines practical text classification with an explanation of precision, recall, and the confusion matrix. The motivation is to build a working course project that connects data preparation, algorithm selection, and a usable interface, and to understand which trade-offs its results actually support.

Overview of the Work Carried Out

The project loads the bundled CSV, identifies label and message columns, cleans the records, and encodes ham as zero and spam as one. It creates a stratified train/test partition, fits one TF-IDF vectorizer on the training text, and transforms both partitions with that mapping. Multinomial Naive Bayes and a linear SVM form the principal baselines. A four-kernel study compares linear, RBF, polynomial, and sigmoid SVMs. A separate ten-candidate internal search chooses parameters using mean spam F1.

The fitted vectorizer and classifiers are saved as joblib artifacts, and the dataset counts, configurations, metrics, confusion matrices, and parameter rankings are recorded in results/summary.json. Flask loads these artifacts and exposes them to the dashboard. The user can enter a message, compare both models, inspect evidence tokens and message statistics, and view the recorded experiment. A retraining route rebuilds the models from the local data when requested.

Scope and Responsible-Use Statement

The implemented system analyzes pasted message text. It does not connect to Gmail or Outlook, move messages between folders, inspect attachments, authenticate senders, or verify that a URL is malicious. Its prediction is a content-based classification aid. Important messages should be reviewed by a person before any consequential action; the project does not implement automatic deletion.

Although the project title refers to email classification, its experiment uses an SMS-derived corpus. Email headers, HTML structure, long messages, and sender information are outside the learned representation. No accuracy claim is made for full email traffic. The local application binds to localhost. Pasted text should be handled carefully, and the development service should not be exposed as a public production system without an appropriate deployment design.

Problem Statement

Case-Study Scenario

Consider a student reviewing a mixture of brief academic messages, personal replies, and unsolicited promotional text. The student wants an initial indication of whether a message resembles spam, together with enough information to understand the result. A false positive can divert attention from a legitimate notice. A false negative leaves unwanted material in the communication stream. This is an illustrative use scenario for evaluating the project, not a claim that the application was deployed to students or tested on their inboxes.

The project addresses this scenario through a message-entry dashboard and a comparison of two model families. The dashboard provides a final SVM label while also showing the Naive Bayes result and whether both models agree. Aggregate evaluation determines how useful that label is on the project corpus; a single convincing demonstration cannot substitute for that evaluation.

Central Case-Study Question

How effectively can our TF-IDF-based Naive Bayes and SVM application identify spam in the project's labelled message corpus, and what do the observed errors imply for a student deciding whether to trust the prediction? The supporting questions concern the effect of kernel choice, the outcome of parameter tuning, and the meaning of the interface's evidence and confidence displays.

Formal Problem Definition

Let D contain pairs of message text and a binary target. For the project, X is the original v2 message column and Y is the original v1 label column. After encoding, $Y = 0$ represents ham and $Y = 1$ represents spam. The feature extractor maps each message to a sparse vector with at most 8,000 dimensions. The classifier learns from the training vectors and predicts the target of another vector produced by the same fitted extractor.

The objective is to obtain useful spam detection while limiting the number of legitimate messages labelled spam. True positives are spam correctly detected; false negatives are spam predicted as ham. False positives are ham predicted as spam, and true negatives are ham correctly accepted. These counts give the evaluation its practical meaning. Precision equals $TP/(TP + FP)$, recall equals $TP/(TP + FN)$, and F1 equals $2TP/(2TP + FP + FN)$.

Objectives that Guide the Study

The first objective is to make the dataset suitable for supervised learning while recording how cleaning changes its size. The second is to use a consistent text representation so that model results are comparable. The third is to compare Naive Bayes, baseline SVM, and the selected SVM using the same test partition and class-specific metrics. The fourth is to examine the four SVM kernels and explain what parameter selection changes.

The application objective is to connect trained models to a working message-entry interface. The study also examines which displayed values are model outputs and which are heuristic summaries. Finally, it identifies evaluation assumptions that must be strengthened before extending claims to a real email setting. The study does not aim to invent a new classifier or claim that the selected algorithms are unique to this project.

Assumptions and Limitations

The labels supplied with the dataset are treated as the target. The experiment assumes that patterns learned from the training partition are informative about the test partition. It does not establish that those patterns transfer to a different language, newer spam campaigns, or full email messages. English stop-word removal and a word vocabulary constrain what the representation can retain.

The available experiment uses one 80/20 partition with `random_state = 42`. The test set contains 131 spam messages, so one additional detected spam message changes recall by about 0.76 percentage points. Small differences between models should therefore be interpreted carefully. Repeated partitions, external validation, inference latency, memory consumption, and calibration quality are not measured in this case study.

There are two additional implementation-specific limitations. Duplicate pairs are removed before whitespace stripping, leaving 11 repeated label/message records after normalization. Six distinct normalized message texts occur in both training and test partitions. Also, TF-IDF is fitted on all outer training text before the two internal validation folds are formed. Those folds therefore share vocabulary and IDF estimates. Both issues weaken evaluation independence and should be corrected in a follow-up experiment.

Dataset and Exploratory Data Analysis

About the Dataset

The working data is stored in `data/spam.csv`, with a recovery copy in `data/spam_canonical.csv`. Both files use the two-column v1/v2 schema. The original collection is associated with the public SMS Spam Collection; the official source is <https://archive.ics.uci.edu/dataset/228/sms+spam+collection>. This report uses counts observed in the bundled project file rather than substituting a catalogue count from another data variant.

Table 1. Actual dataset composition and the reproduced split.

Stage	Records	Ham	Spam
Raw CSV	5,572	4,825	747
Implemented cleaning	5,169	4,516	653
Training partition	4,135	3,613	522
Test partition	1,034	903	131

Purpose of EDA

EDA checks whether the data and evaluation choices fit the classification task. In this project, it helps establish the class balance, message length, and frequency of simple textual characteristics. It also makes duplicate handling visible. Understanding the large ham class explains why a high accuracy value can hide poor spam detection. EDA describes associations in this corpus; it does not prove that a longer message or a capital letter makes a message unwanted.

Preparation Steps Actually Performed

The loader accepts supported label/message column pairs and selects the two relevant columns. It removes missing label or text values, then removes duplicates on the original label/message values. Labels are lowercased and stripped, message text is stripped, unsupported labels are filtered out, and empty messages are excluded. This produces 5,169 records. The raw-minus-clean difference is 403 rows; the bundled file's duplicates account for that difference.

For modelling, labels become integers and the text remains a string input. The split is stratified so that both partitions retain the class imbalance. The fitted vectorizer performs lowercase conversion, Unicode accent stripping, English stop-word removal, and word tokenization. It creates unigrams and bigrams, uses sublinear term frequency, and normalizes feature vectors. No separate stemming step, SMOTE oversampling, or numerical-feature concatenation is present in the inspected training implementation.

For the EDA in this report, character count, whitespace-separated word count, digit count, uppercase count, and the selected punctuation count were recalculated from the cleaned messages. Punctuation here counts only `!`, `?`, `$`, `£`, `₹`, and `%`. These derived columns support the following statistical comparisons and remain separate from the model's TF-IDF features. Stating the definitions makes the reported values reproducible.

EDA Exploration - Implementation

The cleaned corpus contains 87.37% ham and 12.63% spam. The exploratory calculations group messages by their actual labels and compare descriptive values. No missing or empty message remains after the implemented cleaning. Figure 1 shows class counts and the median character count, avoiding any implication that the majority class alone is a useful spam detector.

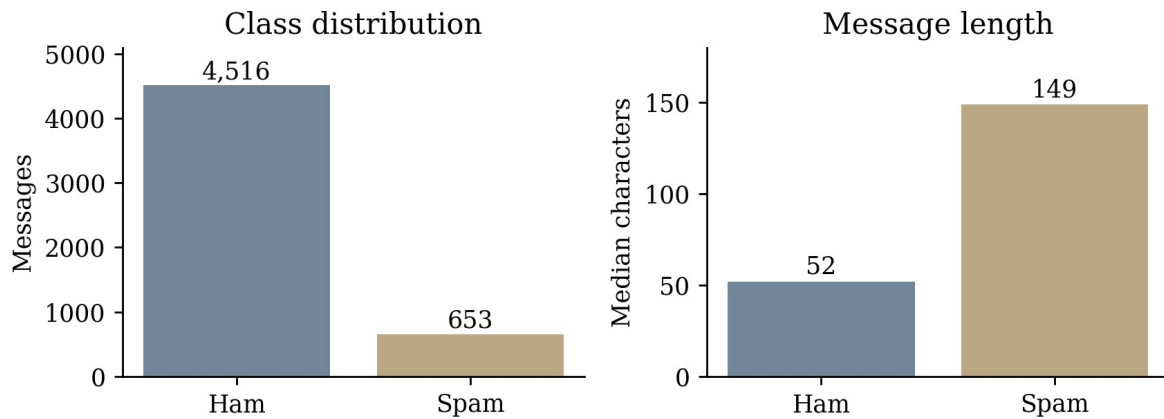


Figure 1. Class composition and median message length calculated from the bundled cleaned corpus.

Table 2. Mean textual characteristics. Word counts use whitespace splitting.

Characteristic	Ham mean	Spam mean
Characters	70.42	137.84
Words	14.13	23.68
Digits	0.29	15.45
Uppercase characters	3.90	15.25
Selected punctuation	0.47	1.36

Spam messages are longer on average and contain more digits, uppercase characters, and selected punctuation in this corpus. Median character counts are 52 for ham and 149 for spam. These differences are descriptive class associations. They do not establish that a fixed length threshold would classify accurately; legitimate messages can also be long or contain a phone number.

Pearson correlation was calculated only between the engineered numerical features. Character and word counts have correlation 0.97, which is expected because both describe message size. Character count and digit count have correlation 0.38. Raw message strings cannot meaningfully be treated as ordinary numerical variables for Pearson correlation. The numeric analysis is an EDA aid and is not used to claim a relationship between a particular word and a causal outcome.

Summary of EDA Findings

The class imbalance makes precision, recall, and F1 necessary. Always predicting ham would score 87.33% accuracy on the test set and detect no spam. Text length and surface statistics differ between classes, but the implemented models use lexical TF-IDF evidence. The residual duplicates show why preparation order matters: normalization should precede deduplication in a stricter follow-up study. Together, the EDA supports the evaluation design while identifying limits that a polished interface alone cannot resolve.

Algorithm Selection and Implementation

Feature Extraction and Model Selection

TF-IDF was selected to represent short messages as sparse numerical vectors without requiring a neural network or external language-model service. The implementation uses `TfidfVectorizer` with `ngram_range = (1, 2)`, `max_features = 8000`, `sublinear_tf = True`, `stop_words = english`, `strip_accents = unicode`, and `min_df = 1`. The extractor is fitted on the outer training partition and reused for the test text and user input. The saved experiment contains 8,000 features.

Multinomial Naive Bayes provides a probabilistic baseline. Its $\alpha = 0.5$ smoothing reduces the problem of zero class-specific feature weights. Prediction combines the learned class prior with weighted log feature evidence. The independence assumption simplifies estimation but does not capture all dependencies between words. TF-IDF features are nonnegative, allowing their use with this estimator.

The baseline SVM uses a linear kernel and $C = 1$. SVM attempts to separate the classes through a decision boundary in the feature representation. The kernel study also evaluates RBF, polynomial, and sigmoid settings. The polynomial degree is three in the initial kernel comparison. No Logistic Regression classifier is part of the analysed project.

Kernel Study and Parameter Tuning

A bounded grid contains ten candidates: three linear settings with $C = 0.1, 1$, and 10 ; two RBF settings with $C = 1$ or 10 and $\gamma = \text{scale}$; another RBF setting with $C = 10$ and $\gamma = 0.1$; two polynomial settings with degrees two and three; and two sigmoid settings with $C = 1$ or 10 . Polynomial and sigmoid settings use $\gamma = \text{scale}$.

The search uses two shuffled stratified validation folds with `random_state = 42`. Candidate results are ranked by mean spam F1, followed by recall, precision, and accuracy as tie-breakers. RBF with $C = 10$ and $\gamma = 0.1$ is selected with mean validation F1 = 88.82%. That candidate is refitted on the full outer training representation. Linear $C = 1$ has mean validation F1 = 88.03%. These values describe internal validation, not the final test result.

Table 3. Test F1 from the predefined $C = 1$ kernel study.

Kernel	Spam F1	Interpretation
Linear	92.31%	Strongest F1 at these settings
RBF	89.92%	High precision with more missed spam
Polynomial	57.61%	Low recall at the default boundary
Sigmoid	91.13%	Close to linear F1 in this partition

The tuned model's label does not imply that it must outperform the baseline on every test partition. The linear baseline has a higher test F1 in this experiment. Kernel performance also depends on the representation, regularization, and decision threshold. These results support a comparison of the recorded configurations rather than a universal ranking of kernel families.

Training Workflow and Application Implementation

The code separates responsibilities into modules. `ml/data.py` validates and cleans the corpus; `ml/training.py` creates the split, vectorizer, model fits, kernel study, and parameter search. `ml/evaluation.py` calculates classification measures. `ml/inference.py` loads artifacts and produces the message-level response, while the runtime and artifact modules track compatibility. `app.py` connects these operations to Flask routes. Templates and static CSS/JavaScript provide the dashboard.

Table 4. Implemented application routes and their role.

Route	Method	Purpose
/	GET	Serve the message-analysis dashboard
/api/health	GET	Report data and model readiness
/api/summary	GET	Return saved experiment metrics
/api/analyze	POST	Classify a JSON text input
/api/retrain	POST	Rebuild local training artifacts

The analysis route accepts a nonempty text string of at most 15,000 characters. The inference module normalizes whitespace and uses the saved vectorizer. It returns Naive Bayes and selected-SVM predictions, agreement, token evidence, a heuristic message profile, and message statistics. If labels disagree, the final verdict uses SVM; if they agree, that label is returned. The final verdict is therefore always the selected SVM prediction, not a voting ensemble.

Evaluation of the Implemented Models

Table 5. Results on the same 1,034-record test partition. Spam is the positive class.

Metric	Naive Bayes	Linear SVM	Tuned RBF SVM
Accuracy	97.49%	98.16%	98.07%
Precision	99.07%	98.28%	97.44%
Recall	80.92%	87.02%	87.02%
F1	89.08%	92.31%	91.94%
False positives	1	2	3
False negatives	25	17	17

The packaged models were loaded and used on the reconstructed test partition. All three confusion matrices matched the summary: Naive Bayes has $TN = 902$ and $TP = 106$; linear SVM has $TN = 901$ and $TP = 114$; tuned RBF has $TN = 900$ and $TP = 114$. Each matrix includes the same 903 actual ham and 131 actual spam messages. This check establishes consistency among the dataset, saved models, and reported results without representing it as a new training experiment.

Interpretation of Results and Interface Evidence

Both SVM configurations detect eight more spam messages than Naive Bayes. Linear SVM creates one extra false positive relative to Naive Bayes, while tuned RBF creates two. The tuned classifier has one more total error than linear SVM. These counts make the case-study trade-off visible: stronger spam recall accompanies a small increase in legitimate messages marked as spam.

The polynomial kernel illustrates why ROC-AUC and F1 should be interpreted separately. It achieves about 99.62% ranking AUC but detects only 53 spam messages and misses 78 at its default boundary. A strong ordering of decision scores does not guarantee a suitable classification threshold. The experiment therefore supports class-specific evaluation rather than reliance on one large percentage.

The token-evidence view ranks present features by their TF-IDF weight multiplied by the difference between spam and ham log feature probabilities in Naive Bayes. It explains feature evidence for Naive Bayes, not the nonlinear SVM. SVM decision strength is calculated as the absolute margin divided by one plus the absolute margin. This is a visual transformation, not a calibrated probability of correctness. Keyword profile scores are heuristic substring counts and are not learned model outputs.

If a false positive is assigned a cost of λ and a false negative a cost of one, the recorded linear-SVM loss is $2\lambda + 17$ and Naive Bayes loss is $\lambda + 25$. Their break-even point is $\lambda = 8$. This simple calculation demonstrates how a person's error preferences can affect model choice. It does not measure real user preferences or establish a recommended operating policy.

Conclusion

Our case study connects an actual labelled dataset, reproducible preprocessing, TF-IDF features, classical classifiers, and a working Flask interface. Linear SVM gives the strongest test F1 among the principal models, while Naive Bayes gives fewer false spam alarms. Cross-validation selects an RBF configuration, but its test score remains slightly below the baseline. The project's main learning outcome is understanding those differences through the error counts and the implemented workflow.

The evidence supports a short-message classification demonstration. Independent email validation, normalization before deduplication, and vectorizer fitting inside each validation fold are the next steps needed for stronger conclusions. The interface should retain a distinction between predictions, model-specific evidence, and heuristic profiles. The project demonstrates supervised learning and model comparison without claiming a new algorithm or a production email-security service.

Signature of the Team members

K. Pranav _____ Abhay Tripathi _____